6

演習 6　外部参照と複雑な SQL 操作

6

演習 6　外部参照と複雑な SQL 操作

演習の目標

- 外部参照を持つエンティティを定義し、テーブル間のリレーション（1 対多、多対 1）を正しく設定できる
- Spring Data JPA を用いて、外部参照先のデータを含む検索・登録処理を実装できる
- JPQL を使って柔軟な検索条件を指定し、実務に近い形の複雑な検索を行える

演習の概要

この章では、第 6 章で学習した「外部参照のあるエンティティの構成方法」や「JPQL による実践的な検索処理」を確認します。

- location テーブルを新たに作成し、employee テーブルと外部参照で紐付ける
- Employee エンティティと Location エンティティのリレーションを構築
- 外部参照を持つ状態での全件検索・主キー検索・登録処理の実装
- JPQL を用いた条件検索（給与と勤務地）を実装

演習 6 外部参照と複雑な SQL 操作

演習の目標

- 外部参照を持つエンティティを定義し、テーブル間のリレーション（ 1 対多、多対 1）を正しく設定できる
- Spring Data JPA を用いて、外部参照先のデータを含む検索・登録処理を実装できる
- JPQL を使って柔軟な検索条件を指定し、実務に近い形の複雑な検索を行える

演習の概要

この章では、第 6 章で学習した「 外部参照のあるエンティティの構成方法」や「 JPQL による実践的な検索処理」を確認します。

- location テーブルを新たに作成し、employee テーブルと外部参照で紐付ける
- Employee エンティティと Location エンティティのリレーションを構築
- 外部参照を持つ状態での全件検索・主キー検索・登録処理の実装
- JPQL を用いた条件検索(給与と勤務地）を実装

予想所要時間

演習 6.1　外部参照を定義したエンティティの修正：10 分

演習 6.2　全件検索で外部参照の内容を表示：10 分

演習 6.3　主キー検索で外部参照の内容を表示：10 分

演習 6.4　外部参照付きの登録処理の実装：15 分

演習 6.5　登録画面の作成（セレクトメニュー）：15 分

演習 6.6　JPQL による条件検索（給与＋勤務地）：20 分

■追加データベース情報

※データベースは、既に構築済みです。

src/resources/application.properties を確認してください。

＜接続情報＞

RDBMS 名	MySQL8.0
データベース名	spring_exercise
ユーザ名	spring_exercise_user
パスワード	password

＜employee テーブル＞

No	論理名称	物理名称	データ型	桁数	制約	備考
1	社員番号	id	INT	–	PRIMARY KEY AUTO_INCREMENT	自動採番
2	パスワード	password	VARCHAR	–	–	–
3	社員名	name	VARCHAR	–	–	–

演習 6 外部参照と複雑な SQL 操作

予想所要時間

演習 6.1　外部参照を定義したエンティティの修正：10 分

演習 6.2　全件検索で外部参照の内容を表示：10 分

演習 6.3　主キー検索で外部参照の内容を表示：10 分

演習 6.4　外部参照付きの登録処理の実装：15 分

演習 6.5　登録画面の作成（セレクトメニュー）：15 分

演習 6.6　JPQL による条件検索（給与＋勤務地）：20 分

■追加データベース情報

※データベースは、既に構築済みです。

src/resources/application.properties を確認してください。

＜接続情報＞

RDBMS 名	MySQL8.0
データベース名	spring_exercise
ユーザ名	spring_exercise_user
パスワード	password

＜employee テーブル＞

No	論理名称	物理名称	データ型	桁数	制約	備考
1	社員番号	id	INT	–	PRIMARY KEY AUTO_INCREMENT	自動採番
2	パスワード	password	VARCHAR	–	–	–
3	社員名	name	VARCHAR	–	–	–

4	給与	salary	INT	–	–	
5	勤務地番号	location_id	INT		FOREIGN KEY	外部キー

<Employee テーブルの初期データー>

ID	password	name	salary	location_id
1001	password	山田太郎	230000	1
1002	password	鈴木一郎	230000	2
1003	password	田中花子	300000	3
1004	password	山田次郎	400000	1
1005	password	高橋美智子	500000	1
1006	password	平恵	270000	5
1007	password	佐々木舞	330000	4
1008	password	中村翔	350000	3

＜location テーブル＞

No	論理名称	物理名称	データ型	桁数	制約	備考
1	勤務地番号	id	INT	–	PRIMARY KEY AUTO_INCREMENT	自動採番
2	勤務地名	name	VARCHAR	–	–	–

< location テーブルの初期データ>

id	name
1	tokyo
2	osaka
3	fukuoka
4	sapporo
5	okinawa

演習 6 外部参照と複雑な SQL 操作

4	給与	salary	INT	–	–	
5	勤務地番号	location_id	INT		FOREIGN KEY	外部キー

<Employee テーブルの初期データー>

ID	password	name	salary	location_id
1001	password	山田太郎	230000	1
1002	password	鈴木一郎	230000	2
1003	password	田中花子	300000	3
1004	password	山田次郎	400000	1
1005	password	高橋美智子	500000	1
1006	password	平恵	270000	5
1007	password	佐々木舞	330000	4
1008	password	中村翔	350000	3

＜location テーブル＞

No	論理名称	物理名称	データ型	桁数	制約	備考
1	勤務地番号	id	INT	–	PRIMARY KEY AUTO_INCREMENT	自動採番
2	勤務地名	name	VARCHAR	–	–	–

< location テーブルの初期データ>

id	name
1	tokyo
2	osaka
3	fukuoka
4	sapporo
5	okinawa

演習6.1. エンティティ・リポジトリの追加と修正

a. Location エンティティを確認してください。

- jp.co.trainocate.enshu.entity パッケージに、Location.java はすでに作成済みです。
- 以下の内容を確認してください。
    - クラスに@Entity、@Table アノテーションを付けること
    - id フィールドを主キー（@Id）に設定し、自動採番（@GeneratedValue）にすること

🔎【ヒント】

- テキスト 6.1 の【作業手順②】を参考にしてください。
- 参照関係の定義は、後ほど行いますので、不要です。

b. Employee エンティティを修正してください。

- jp.co.trainocate.enshu.entity パッケージに、Employee.java はすでに作成済みです。
- 以下の内容を追記してください。
    - Location 型の location フィールドを追加すること
    - @ManyToOne と @JoinColumn を使い、location_id カラムと外部参照の関連付けを行うこと
        - @JoinColumn(name = "location_id")と指定してください。

🔎【ヒント】

- テキスト 6.1 の【作業手順③】を参考にしてください。

c. Location エンティティに参照元（Employee）を持たせましょう。

- jp.co.trainocate.enshu.entity パッケージの Location.java を修正してください。
- 以下の内容を追記してください。
    - Employee エンティティを格納するリストフィールドを作成すること
    - @OneToMany(mappedBy = "location") アノテーションを付けること
        - mappedBy には、Employee エンティティ側で定義したフィールド名（"location"）を指定してください。

演習 6 外部参照と複雑な SQL 操作

演習6.1. エンティティ・リポジトリの追加と修正

a. Location エンティティを確認してください。

- jp.co.trainocate.enshu.entity パッケージに、Location.java はすでに作成済みです。
- 以下の内容を確認してください。
    - クラスに@Entity、@Table アノテーションを付けること
    - id フィールドを主キー(@Id) に設定し、自動採番(@GeneratedValue) にすること

【 ヒント】

- テキスト 6.1 の【 作業手順②】を参考にしてください。
- 参照関係の定義は、後ほど行いますので、不要です。

b. Employee エンティティを修正してください。

- jp.co.trainocate.enshu.entity パッケージに、Employee.java はすでに作成済みです。
- 以下の内容を追記してください。
    - Location 型の location フィールドを追加すること
    - @ManyToOne と @JoinColumn を使い、location_id カラムと外部参照の関連付けを行うこと
        - @JoinColumn(name = "location_id")と指定してください。

【 ヒント】

- テキスト 6.1 の【 作業手順③】を参考にしてください。

c. Location エンティティに参照元(Employee）を持たせましょう。

- jp.co.trainocate.enshu.entity パッケージの Location.java を修正してください。
- 以下の内容を追記してください。
    - Employee エンティティを格納するリストフィールドを作成すること
    - @OneToMany(mappedBy = "location") アノテーションを付けること
        - mappedBy には、Employee エンティティ側で定義したフィールド名("location") を指定してください。

- フィールド名は、employeeList としてください。

🔎【ヒント】

- テキスト 6.1 の【作業手順②】を参考にしてください。
- 「1 つの勤務地に複数の社員が所属する」関係を表現します。
- リスト型は java.util.List をインポートしてください。

d. Location エンティティ用の Repository インタフェースを確認してください。

- jp.co.trainocate.enshu.repository パッケージ内に、LocationRepository.java が作成されています。確認してください。
- Location エンティティを対象とした Repository です。

🔎【ヒント】

- JpaRepository を継承することで、findAll(), findById(), save() などが自動的に使えるようになります。

演習 6 外部参照と複雑な SQL 操作

- フィールド名は、employeeList としてください。

【ヒント】

- テキスト 6.1 の【作業手順②】を参考にしてください。
- 「1 つの勤務地に複数の社員が所属する」関係を表現します。
- リスト型は java.util.List をインポートしてください。

d. Location エンティティ用の Repository インタフェースを確認してください。

- jp.co.trainocate.enshu.repository パッケージ内に、LocationRepository.java が作成されています。確認してください。
- Location エンティティを対象とした Repository です。

【ヒント】

- JpaRepository を継承することで、findAll(), findById(), save() などが自動的に使えるようになります。

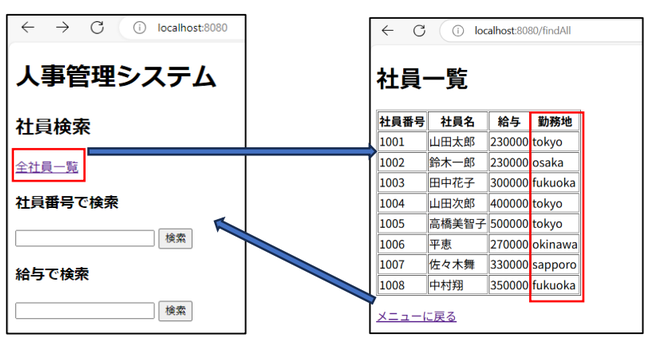
52

演習6.2. 外部参照を持つデータの全件検索

■Employee と Location のリレーションを利用して、全件検索（findAll）を行います。

■取得したリストデータを、一覧表示ページに出力できるようにします。

■Controller 側と HTML 側で、役割に応じた修正を行います。



a. Controller クラスの全件検索処理(findAll メソッド)を完成してください。

- jp.co.trainocate.enshu.controller.EmpController クラスに追記してください。
- Repository を使って全社員データを取得し、Model に格納してください。
- Model には、リストを"empList"という名前で格納してください。
- 遷移先の HTML ファイルは、emp_list.html としてください。

🔎【ヒント】

- Repository の findAll()メソッドを利用します。
- location の name も一覧で表示できるようにデータを準備します。

b. employee_list.html に一覧出力する処理を追加してください。

- すでに用意されている emp_list.html ファイルを修正してください。
- empList リストから、社員番号、社員名、給与、**勤務地名**を表示してください。
- Thymeleaf の th:each 構文を利用してください。

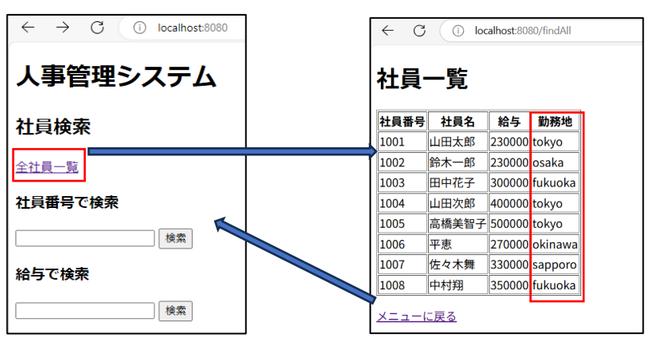
演習 6 外部参照と複雑な SQL 操作

演習6.2. 外部参照を持つデータの全件検索

■Employee と Location のリレーションを利用して、全件検索(findAll)を行います。

■取得したリストデータを、一覧表示ページに出力できるようにします。

■Controller 側と HTML 側で、役割に応じた修正を行います。



a. Controller クラスの全件検索処理(findAll メソッド)を完成してください。

- jp.co.trainocate.enshu.controller.EmpController クラスに追記してください。
- Repository を使って全社員データを取得し、Model に格納してください。
- Model には、リストを"empList"という名前で格納してください。
- 遷移先の HTML ファイルは、emp_list.html としてください。

【ヒント】

- Repository の findAll()メソッドを利用します。
- location の name も一覧で表示できるようにデータを準備します。

b. employee_list.html に一覧出力する処理を追加してください。

- すでに用意されている emp_list.html ファイルを修正してください。
- empList リストから、社員番号、社員名、給与、**勤務地名**を表示してください。
- Thymeleaf の th:each 構文を利用してください。

🔎【ヒント】

- テキスト 6.2 の【作業手順②】を参考にしてください。

動作確認：http://localhost:8080/
「全社員一覧」リンクをクリックし、社員リストが表示されるか確認してください。

演習6.3. 社員番号を使った個別検索

■社員番号（id）で 1 件検索できる機能を作ります。

■検索結果を画面に正しく出力できるようにします。

a. Controller クラスの主キー検索処理(findById メソッド)を完成してください。

- jp.co.trainocate.enshu.controller.EmpController クラスに追記してください。
- Repository を使って、指定された id の社員データを取得してください。
- 取得した Employee オブジェクトを"emp"という名前で Model に格納してください。
- 遷移先は emp.html にしてください。

🔎【ヒント】

- URL マッピングは /showFindById としてください（3 章と同じ形式）。

b. emp.html に 1 件分のデータを出力するコードを追加してください。

- すでに用意されている emp.html ファイルを修正してください。
- 社員番号、社員名、給与、**勤務地名**を表示するようにしてください。

🔎【ヒント】

- データ 1 件だけなので、th:each は不要です。

動作確認：http://localhost:8080/
「社員番号で検索」に、「1001」を入力し、検索します。社員が表示されるか確認してください。

【ヒント】

・ テキスト 6.2 の【作業手順②】を参考にしてください。

動作確認：http://localhost:8080/
「全社員一覧」リンクをクリックし、社員リストが表示されるか確認してください。

演習6.3. 社員番号を使った個別検索

■社員番号（id）で 1 件検索できる機能を作ります。

■検索結果を画面に正しく出力できるようにします。

a. Controller クラスの主キー検索処理(findById メソッド)を完成してください。

・ jp.co.trainocate.enshu.controller.EmpController クラスに追記してください。
・ Repository を使って、指定された id の社員データを取得してください。
・ 取得した Employee オブジェクトを"emp"という名前で Model に格納してください。
・ 遷移先は emp.html にしてください。

【ヒント】

・ URL マッピングは /showFindById としてください（3 章と同じ形式）。

b. emp.html に 1 件分のデータを出力するコードを追加してください。

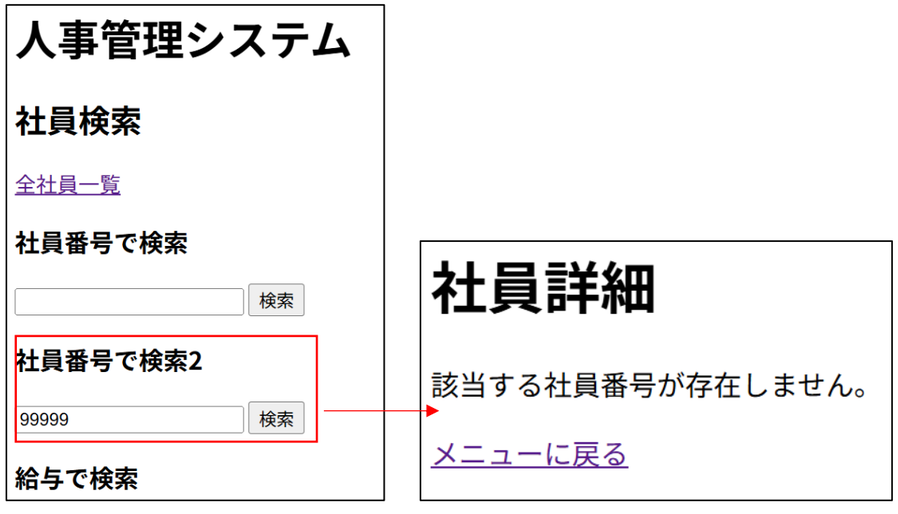
・ すでに用意されている emp.html ファイルを修正してください。
・ 社員番号、社員名、給与、**勤務地名**を表示するようにしてください。

【ヒント】

・ データ 1 件だけなので、th:each は不要です。

動作確認：http://localhost:8080/
「社員番号で検索」に、「1001」を入力し、検索します。社員が表示されるか確認してください。

演習6.4. 【オプション】存在しない社員番号を考慮する個別検索



■findById(id).orElse(null) を使って、存在しない値でも安全な主キー検索を実装します。

■社員が見つからない場合、エラーメッセージを表示する処理を実装します。

a. Controller クラスに存在しない値でも安全な主キー検索メソッドを完成してください。

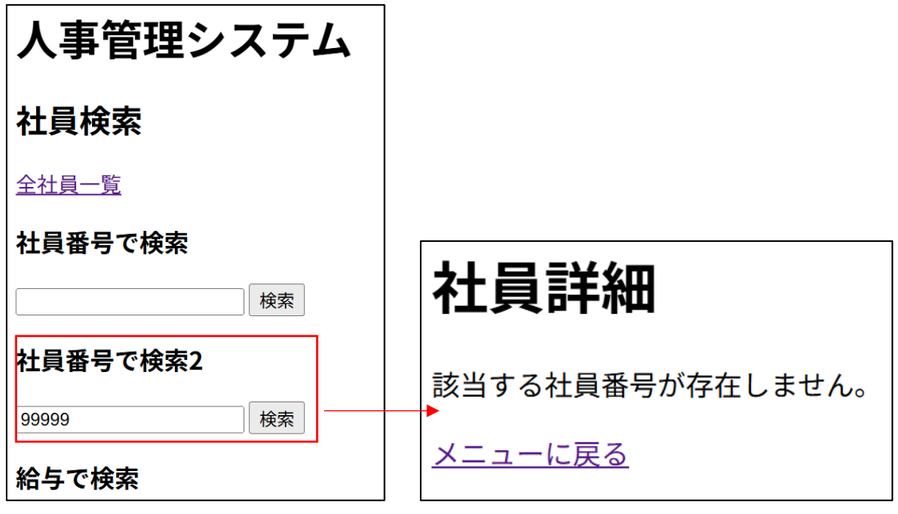
- jp.co.trainocate.enshu.controller.EmpController に新しくメソッドを追加してください。
- Repository の findById(id).orElse(null) を使って検索してください。
- 取得できた Employee オブジェクトを "emp" という名前で Model に格納してください。
- 遷移先は **emp2.html** にしてください。

🔎【ヒント】

- if 文などで null 判定は不要です。orElse(null)により、null かオブジェクトかが自動で取得できます。
- テキスト 6.2.1 の【作業手順①】を参考にしてください。

演習 6 外部参照と複雑な SQL 操作

演習6.4.【 オプション】存在しない社員番号を考慮する個別検索



■findById(id).orElse(null) を使って、存在しない値でも安全な主キー検索を実装します。

■社員が見つからない場合、エラーメッセージを表示する処理を実装します。

a. Controller クラスに存在しない値でも安全な主キー検索メソッドを完成してください。

- jp.co.trainocate.enshu.controller.EmpController に新しくメソッドを追加してください。
- Repository の findById(id).orElse(null) を使って検索してください。
- 取得できた Employee オブジェクトを "emp" という名前で Model に格納してください。
- 遷移先は **emp2.html** にしてください。

【 ヒント】

- if 文などで null 判定は不要です。orElse(null)により、null かオブジェクトかが自動で取得できます。
- テキスト 6.2.1 の【 作業手順①】を参考にしてください。

b. emp.html でデータ存在チェックを追加してください。

- すでにある **emp2.html** ファイルに、表示切り替えの処理を追記してください。
- 社員データが存在しない場合は、「該当する社員番号は存在しません。」と表示してください。
- th:if と th:unless を使いましょう。

🔎【ヒント】

- th:if="${emp == null}" と th:unless="${emp == null}" を使い、切り替えます。
- テキスト 6.2.1 の【作業手順②】を参考にしてください。

動作確認：http://localhost:8080/
「社員番号で検索 2」に、「9999」を入力し、検索します。
「該当する社員情報は存在しません。」が表示されることを確認します。

演習 6 外部参照と複雑な SQL 操作

b. emp.html でデータ存在チェックを追加してください。

- すでにある **emp2.html** ファイルに、表示切り替えの処理を追記してください。
- 社員データが存在しない場合は、「該当する社員番号は存在しません。」と表示してください。
- th:if と th:unless を使いましょう。

【ヒント】

- th:if="${emp == null}" と th:unless="${emp == null}" を使い、切り替えます。
- テキスト 6.2.1 の【作業手順②】を参考にしてください。

動作確認：http://localhost:8080/
「社員番号で検索 2」に、「9999」を入力し、検索します。
「該当する社員情報は存在しません。」が表示されることを確認します。

演習6.5. 社員情報の登録



■問題構成

小問	内容
a	入力画面へ遷移する Controller メソッドの作成
b	location テーブルのリストを使って、<select> メニューを表示
c	EmpForm に locationId を追加
d	登録処理の実装（save 処理）
e	外部キーの登録時に必要なエンティティ側の設定を確認

a. 入力画面に遷移するためのメソッドを EmpController に追加してください。

- 遷移先の HTML ファイルは emp_input.html にしてください。
- URL マッピングは /showInputView にしてください。
- 画面に <select> を表示するために、Location の全件を取得し、"locations" という名前で Model に格納してください。

演習 6 外部参照と複雑な SQL 操作

演習6.5. 社員情報の登録



■問題構成

小問	内容
a	入力画面へ遷移する Controller メソッドの作成
b	location テーブルのリストを使って、<select> メニューを表示
c	EmpForm に locationId を追加
d	登録処理の実装(save 処理)
e	外部キーの登録時に必要なエンティティ側の設定を確認

a. 入力画面に遷移するためのメソッドを EmpController に追加してください。

- 遷移先の HTML ファイルは emp_input.html にしてください。
- URL マッピングは /showInputView にしてください。
- 画面に <select> を表示するために、Location の全件を取得し、"locations" という名前で Model に格納してください。

🔎【ヒント】

- LocationRepository から findAll() を使って、Location のリストを取得できます

b. emp_input.html に、location を選択できるセレクトボックスを作成してください。

- Form 内で、社員名、給与、パスワード、勤務地（location）を入力できるようにしてください。
- Location の選択肢は、locations というリストを th:each でループさせて出力してください。
- 選択された勤務地は、**name="locationId"** でサーバーへ送信してください。

🔎【ヒント】

- テキスト 6.5 の【作業手順①】を参考にしてください。

動作確認：http://localhost:8080/
「社員データ登録」リンクをクリックします。勤務地の選択肢が正しく表示されるのかを確認します。

c. EmpForm に locationId フィールドを追加してください。

- 登録画面で選択された勤務地（locationId）を受け取るために、EmpForm にフィールドを追加してください。
- データ型は Integer としてください。
- name 属性は "locationId" ですので、フォームのフィールド名と一致させてください。
- @Data アノテーションがすでに付いているため、getter/setter は不要です。

🔎【ヒント】

- select タグの name="locationId" に合わせたフィールド名にしてください。
- salary と同じように、Integer 型で定義すれば OK です。

演習 6 外部参照と複雑な SQL 操作

【ヒント】

- LocationRepository から findAll() を使って、Location のリストを取得できます

b. emp_input.html に、location を選択できるセレクトボックスを作成してください。

- Form 内で、社員名、給与、パスワード、勤務地(location）を入力できるようにしてください。
- Location の選択肢は、locations というリストを th:each でループさせて出力してください。
- 選択された勤務地は、**name="locationId"** でサーバーへ送信してください。

【ヒント】

- テキスト 6.5 の【 作業手順①】を参考にしてください。

動作確認：http://localhost:8080/
「 社員データ登録」リンクをクリックします。勤務地の選択肢が正しく表示されるのかを確認します。

c. EmpForm に locationId フィールドを追加してください。

- 登録画面で選択された勤務地(locationId）を受け取るために、EmpForm にフィールドを追加してください。
- データ型は Integer としてください。
- name 属性は "locationId" ですので、フォームのフィールド名と一致させてください。
- @Data アノテーションがすでに付いているため、getter/setter は不要です。

【ヒント】

- select タグの name="locationId" に合わせたフィールド名にしてください。
- salary と同じように、Integer 型で定義すれば OK です。

58

d. EmpController に登録処理メソッド insertEmp を実装してください。

- EmpForm から受け取った値を使って、Employee エンティティを生成してください。
- Employee エンティティには Location location という参照フィールドがあるため、locationId の値をそのまま setLocation() することはできません。
- そのため、**Employee エンティティを開いて、 locationId を別フィールドとして追加し、登録時はそちらを利用してください。**
- リポジトリの save() メソッドで DB に登録してください。
- 登録後は、 /findAll にリダイレクトしてください。

🔎【ヒント】

- Employee エンティティ内の location フィールドは Location 型です。一方、Form からは Integer 型の locationId が送られてくるため、直接代入はできません。
  代わりに locationId を登録用に持たせておき、 location フィールドには insertable = false, updatable = false を指定しておく必要があります。
- **テキスト 6.5 の【作業手順④・⑤】を参考にしてください。**

フィールド	説明
location（Location 型）	外部テーブルと結びつけるためのオブジェクト。SELECT では自動で JOIN してくれるが、登録・更新には使わない。
locationId（Integer 型）	フォームから渡された実際の整数値を使って登録・更新を行うためのフィールド。DB のカラムに直接反映される。
insertable = false, updatable = false	location（参照オブジェクト）を登録や更新に使わないように Hibernate に明示するための設定。

動作確認：http://localhost:8080/
「社員データ登録」リンクをクリックし、任意のデータを入れ、登録します。正しく登録されるのかを確認してください。

演習 6 外部参照と複雑な SQL 操作

d. EmpController に登録処理メソッド insertEmp を実装してください。

- EmpForm から受け取った値を使って、Employee エンティティを生成してください。
- Employee エンティティには Location location という参照フィールドがあるため、locationId の値をそのまま setLocation() することはできません。
- そのため、**Employee エンティティを開いて、 locationId を別フィールドとして追加し、登録時はそちらを利用してください。**
- リポジトリの save() メソッドで DB に登録してください。
- 登録後は、 /findAll にリダイレクトしてください。

【ヒント】

- Employee エンティティ内の location フィールドは Location 型です。一方、Form からは Integer 型の locationId が送られてくるため、直接代入はできません。
  代わりに locationId を登録用に持たせておき、 location フィールドには insertable = false, updatable = false を指定しておく必要があります。
- **テキスト 6.5 の【 作業手順④・⑤】を参考にしてください。**

フィールド	説明
location（ Location 型）	外部テーブルと結びつけるためのオブジェクト。SELECT では自動で JOIN してくれるが、登録・更新には使わない。
locationId（ Integer 型）	フォームから渡された実際の整数値を使って登録・更新を行うためのフィールド。DB のカラムに直接反映される。
insertable = false, updatable = false	location（ 参照オブジェクト）を登録や更新に使わないように Hibernate に明示するための設定。

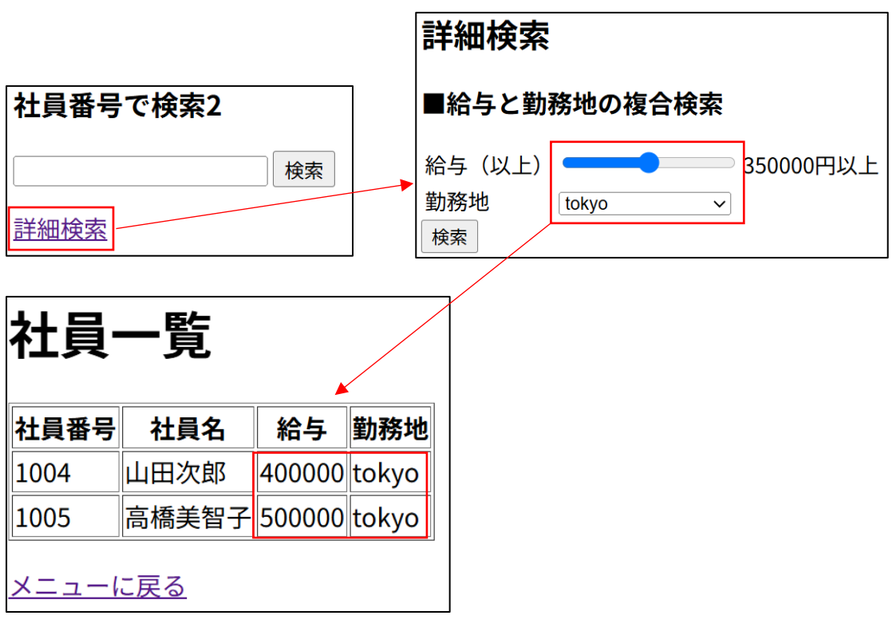
動作確認：http://localhost:8080/
「 社員データ登録」リンクをクリックし、任意のデータを入れ、登録します。正しく登録されるのかを確認してください。

演習6.6. JPQL を使った検索(給与+勤務地)

給与と勤務地の AND 検索は、SQL 文にすると以下のようになります。

```
「SELECT *
FROM employee e  JOIN location l
ON e.location_id = l.id
WHERE e.salary >= ? AND l.name = ?」
```

複雑な検索条件を JPQL で柔軟な検索条件を記述し、給与と勤務地に一致する社員を検索できるようにしましょう。



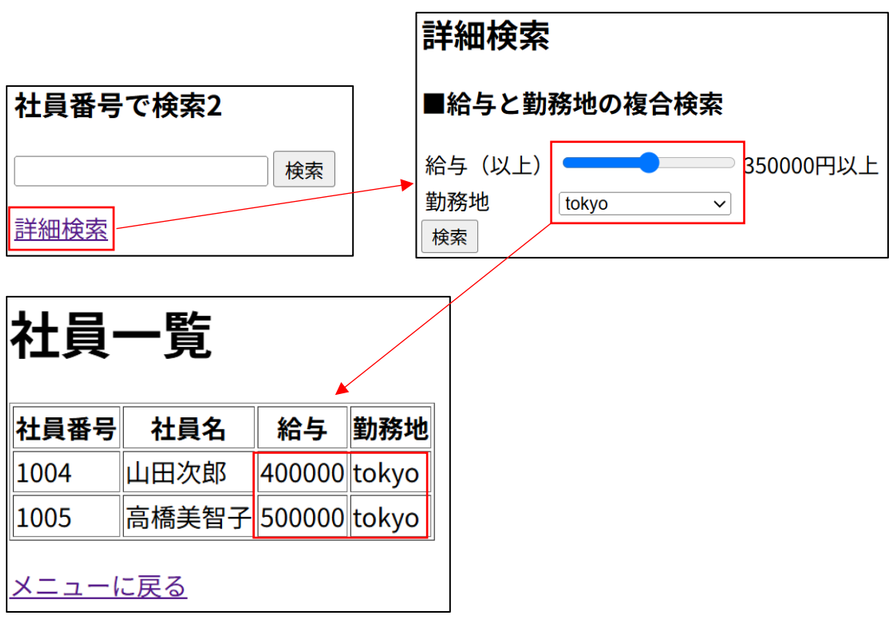
演習 6 外部参照と複雑な SQL 操作

演習6.6. JPQL を使った検索(給与+勤務地)

給与と勤務地の AND 検索は、SQL 文にすると以下のようになります。

```
「SELECT *
FROM employee e  JOIN location l
ON e.location_id = l.id
WHERE e.salary >= ? AND l.name = ?」
```

複雑な検索条件を JPQL で柔軟な検索条件を記述し、給与と勤務地に一致する社員を検索できるようにしましょう。



a. EmpController の showSearchForm メソッドのソースコードを確認してください。

1. 演習 6.5 と同様に、検索画面の表示のためにも勤務地リストの取得が必要です。

b. 入力画面（emp_search.html）が、以下の内容で作成されていることを確認してください。

1. 「/showSearchForm」でアクセスすると検索入力画面が表示される
2. 入力画面には以下の構成がある：

入力項目	内容
給与	type="range" で、給与を 200,000～500,000 の範囲から選べるようにしている
勤務地	DB の Location テーブルから name を取得し、セレクトメニューに表示している

🔎【ヒント】

- range を使うとスライダー式で給与を選べます。
- th:each="loc : ${locationList}" を使って、セレクトメニューに DB の勤務地一覧を表示できます。
- 今いくらに設定されているのかの確認のため、一部 JavaScript のコードが混ざっています。

c. EmpRepository に、給与と勤務地を条件にした JPQL を使った検索メソッドを実装してください。

- 戻り値は List<Employee> としてください。
- メソッド名は findHighSalaryByLocation としてください。
- @Query アノテーションを使って、JPQL を記述してください。
- 引数は int salary、String locationName の 2 つを受け取れるようにしてください。

🔎【ヒント】

- エンティティ名は Employee(JPQL では、**テーブル名ではなくエンティティ名**を使います。)
- location テーブルは @ManyToOne で結ばれているため、e.location.name のように書くことができます。

演習 6 外部参照と複雑な SQL 操作

a. EmpController の showSearchForm メソッドのソースコードを確認してください。

1. 演習 6.5 と同様に、検索画面の表示のためにも勤務地リストの取得が必要です。

b. 入力画面(emp_search.html）が、以下の内容で作成されていることを確認してください。

1. 「 /showSearchForm」でアクセスすると検索入力画面が表示される
2. 入力画面には以下の構成がある：

入力項目	内容
給与	type="range" で、給与を 200,000～500,000 の範囲から選べるようにしている
勤務地	DB の Location テーブルから name を取得し、セレクトメニューに表示している

【 ヒント】

- range を使うとスライダー式で給与を選べます。
- th:each="loc : ${locationList}" を使って、セレクトメニューに DB の勤務地一覧を表示できます。
- 今いくらに設定されているのかの確認のため、一部 JavaScript のコードが混ざっています。

c. EmpRepository に、給与と勤務地を条件にした JPQL を使った検索メソッドを実装してください。

- 戻り値は List<Employee> としてください。
- メソッド名は findHighSalaryByLocation としてください。
- @Query アノテーションを使って、JPQL を記述してください。
- 引数は int salary、String locationName の 2 つを受け取れるようにしてください。

【 ヒント】

- エンティティ名は Employee(JPQL では、テーブル名ではなくエンティティ名を使います。)
- location テーブルは @ManyToOne で結ばれているため、e.location.name のように書くことができます。

- WHERE 句には e.salary >= :salary AND e.location.name = :locationName のように書きます。
- テキストの 6.4.2 の部分を参考にしてください。

d. EmpController に、以下の条件で JPQL による検索を実行するメソッドを追加してください。

内容	詳細
URL	/findHighSalary
HTTP メソッド	GET
引数	int salary, String locationName, Model model
使用するリポジトリメソッド	findHighSalaryByLocation()（演習 6.6.c で実装済み）
HTML ファイル	検索結果は emp_list.html に表示
Model への追加	取得したリストを "empList" のキー名で格納

動作確認：http://localhost:8080/

「詳細検索」リンクをクリックし、給与を「400000 円」以上、勤務地は「tokyo」に設定し、検索してください。2 件のデータが表示されるかを確認してください。

演習 6 外部参照と複雑な SQL 操作

- WHERE 句には e.salary >= :salary AND e.location.name = :locationName のように書きます。
- テキストの 6.4.2 の部分を参考にしてください。

d. EmpController に、以下の条件で JPQL による検索を実行するメソッドを追加してください。

内容	詳細
URL	/findHighSalary
HTTP メソッド	GET
引数	int salary, String locationName, Model model
使用するリポジトリメソッド	findHighSalaryByLocation()(演習 6.6.c で実装済み)
HTML ファイル	検索結果は emp_list.html に表示
Model への追加	取得したリストを "empList" のキー名で格納

動作確認：http://localhost:8080/

「 詳細検索」リンクをクリックし、給与を「 400000 円」以上、勤務地は「 tokyo」に設定し、検索してください。2 件のデータが表示されるかを確認してください。